

Agente Conversacional con Web Scraping, Recuperación Densa y Memoria Persistente para Información Institucional Hospitalaria: Fundación Valle del Lili

Frank Edward Daza González Andrés Fernando López Rendón Viviana Valle Fernández
Técnicas Avanzadas de IA Aplicadas en Modelos de Lenguaje
Universidad Autónoma de Occidente, Cali, Colombia
Docente: Jan Polanco Velasco

Resumen—Este artículo presenta la segunda fase del sistema de asistencia conversacional para la Fundación Valle del Lili, que evoluciona del pipeline RAG léxico (BM25) del Módulo 1 hacia un agente conversacional completo con tres componentes nucleares: recuperación densa en Qdrant con *embeddings*, memoria multi-turno en PostgreSQL y orquestación dinámica con LangGraph. El *web scraping* del corpus institucional, descrito en la fase anterior, mantiene su función como fuente de verdad textual y alimenta el proceso de ingesta hacia el almacén vectorial. El agente orquesta tres herramientas complementarias: *faq_estructurada* (respuestas deterministas sobre un JSON curado), *rag_denso* (recuperación semántica por similitud coseno con diversificación MMR y reranking cruzado opcional) y *listar_estructurado* (agregación y conteo por filtros de *payload* en Qdrant). La transmisión de la respuesta al cliente se realiza mediante eventos SSE extendidos (*pensamiento*, *herramienta*, *token*, *fuentes*, *final*, *error*). La interfaz de usuario se migró de Gradio a una SPA en React 19 con Vite 8, TypeScript 6, Tailwind v4 y shadcn/ui. Se incluye un panel administrativo con recarga en caliente de parámetros LLM y RAG sin reiniciar el servidor, y una suite de evaluación con métricas nDCG, Recall@K, Precision@K y MRR sobre un conjunto de referencia (*golden set*).

Index Terms—agente conversacional, recuperación aumentada por generación, LangGraph, LlamaIndex, Qdrant, PostgreSQL, FastAPI, SSE, React, *web scraping*, modelos de lenguaje grandes, inteligencia artificial

I. INTRODUCCIÓN

El Módulo 1 del proyecto estableció una base sólida: un corpus textual de 40+ páginas extraídas de <https://valledellili.org/> mediante *web scraping*, convertidas a Markdown con metadatos YAML y consultables mediante BM25 [?]. Si bien ese enfoque demostró la viabilidad del RAG léxico, presentó limitaciones inherentes: incapacidad para capturar similitud semántica, recuperación de un único documento sin diversificación y ausencia de memoria conversacional entre turnos.

El Módulo 2 supera estas limitaciones mediante una arquitectura de agente completo [?], [?] que incorpora: (i) recuperación densa con *embeddings* y similitud coseno en Qdrant [?]; (ii) diversificación MMR [?] y reranking con *cross-encoder*; (iii) memoria conversacional multi-turno en PostgreSQL [?];

(iv) orquestación de flujo con LangGraph [?]; y (v) interfaz SPA moderna con React 19 y streaming SSE.

La contribución principal de este trabajo es la demostración de una arquitectura de agente conversacional educativamente completa, que integra múltiples herramientas heterogéneas bajo un grafo de orquestación unificado, con trazabilidad de razonamiento expuesta al usuario final y capacidad de ajuste administrativo en tiempo de ejecución sin reiniciar el servidor.

II. ARQUITECTURA GENERAL DEL SISTEMA

La Figura ?? muestra los componentes del sistema y sus interacciones. El corpus textual, obtenido mediante *web scraping* (Módulo 1), fluye hacia Qdrant a través del script de ingesta. En tiempo de inferencia, cada petición del usuario pasa por FastAPI, que delega al grafo LangGraph, el cual accede a PostgreSQL (memoria) y Qdrant (RAG) según la herramienta elegida.

La Tabla ?? resume el *stack* tecnológico empleado.

Cuadro I
STACK TECNOLÓGICO DEL MÓDULO 2

Capa	Tecnología	Versión
Orquestación	LangGraph	0.2.x
Tools y memoria	LangChain + langchain-postgres	0.3.x
RAG denso	LlamaIndex + qdrant-client	0.12.x
Vector store	Qdrant	v1.18.0
OLTP	PostgreSQL + SQLAlchemy async	17.9 / 2.x
Migraciones	Alembic	1.13.x
Backend HTTP	FastAPI + Uvicorn + sse-starlette	0.115.x
Frontend	React 19 + Vite 8 + TypeScript 6	—
UI components	Tailwind v4 + shadcn/ui	—
Streaming UI	fetch + ReadableStream + Zod	—
Contenedores	Docker Compose	—

III. CORPUS Y PROCESO DE INGESTA

III-A. Continuidad con el Módulo 1

El *web scraping* de <https://valledellili.org/> y la conversión a Markdown con *front matter* YAML descritos

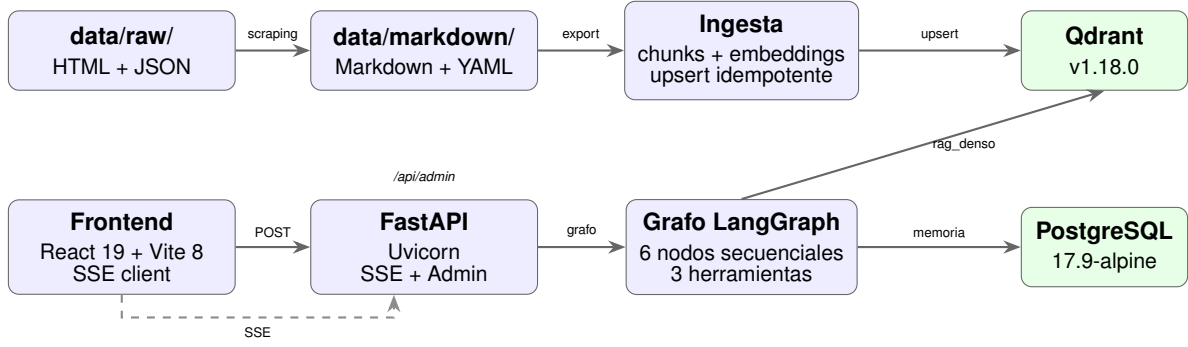


Figura 1. Arquitectura del sistema Módulo 2. El corpus (izquierda) se indexa *offline* en Qdrant (fila superior). En inferencia (fila inferior), Frontend → FastAPI → LangGraph accede a Qdrant (*rag_denso*) y a PostgreSQL (memoria conversacional).

en el Módulo 1 permanecen sin cambios. El directorio `data/markdown/valledellili-org/` (40+ archivos) constituye la fuente de verdad textual del sistema.

III-B. Ingesta en Qdrant

El script `scripts/indexar_corpus_qdrant.py` transforma el corpus textual en vectores y los carga en Qdrant mediante el proceso de la Figura ??.

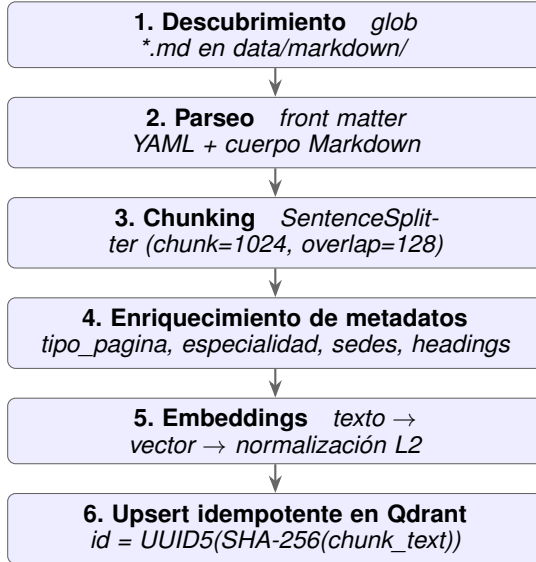


Figura 2. Pipeline de ingestión: de Markdown a puntos vectoriales en Qdrant.

La idempotencia se garantiza mediante el identificador de punto:

$$\text{id_chunk} = \text{UUID5}(\mathcal{N}, \text{SHA256}(\text{chunk_text}))$$

donde $\mathcal{N}$ es un *namespace* fijo del proyecto. Un *upsert* sobre un chunk sin cambios solo reescribe el vector y el *payload* con valores idénticos, sin duplicar datos.

Cada punto en Qdrant almacena en su *payload*: *archivo*, *título*, *source_url*, *tipo_pagina*, *especialidad*, *sedes*, *chunk_index* y un campo *texto* con los primeros 500 caracteres del fragmento.

III-C. Configuración de Embeddings

El sistema soporta dos proveedores de *embeddings* intercambiables (Tabla ??), seleccionables mediante la variable `EMBEDDING_PROVIDER`.

Cuadro II
PROVEEDORES DE EMBEDDINGS SOPORTADOS

Proveedor	Modelo por defecto	Dims	Tipo
OpenAI	text-embedding-3-small	1536	API (nube)
HuggingFace	all-MiniLM-L6-v2	384	Local (CPU)

IV. MÓDULO DE RECUPERACIÓN DENSA

IV-A. Similitud Coseno y Búsqueda en Qdrant

La similitud entre el vector de consulta $\mathbf{q}$ y cada vector de documento $\mathbf{d}_j$ se calcula como:

$$\text{sim}(\mathbf{q}, \mathbf{d}_j) = \frac{\mathbf{q} \cdot \mathbf{d}_j}{\|\mathbf{q}\|_2 \|\mathbf{d}_j\|_2} \quad (1)$$

Los vectores se normalizan a norma unitaria en la ingestión, por lo que la búsqueda se reduce a un producto escalar, lo que optimiza el cómputo en Qdrant. La clase `RecuperadorDenso` (en `src/rag/recuperador_denso.py`) solicita inicialmente $K_0 = \max(K_{\text{ini}}, K)$ candidatos ($K_{\text{ini}} = 20$ por defecto), que luego se filtran y refinan.

IV-B. Filtro por Umbral y Filtros de Payload

Tras obtener los candidatos, se descartan los fragmentos cuya similitud no alcanza el umbral mínimo:

$$\mathcal{F} = \{(\mathbf{d}_j, s_j) : s_j \geq s_{\text{mín}}, \quad s_{\text{mín}} = 0.25\}$$

Adicionalmente, la búsqueda puede restringirse a subconjuntos del corpus mediante filtros sobre el campo *tipo_pagina* del *payload* (e.g. ["institucional"], ["sede"], ["ficha_medico"]), infiriendo la intención de la consulta con heurísticas de expresión regular.

IV-C. Reranking con Cross-Encoder (Opcional)

Cuando `RAG_RERANKER_HABILITADO=true`, los primeros N_r candidatos se puntúan con un *cross-encoder* (modelo por defecto: `BAAI/bge-reranker-base` [?]):

$$\text{score_rerank}(q, d) = \text{CrossEncoder}(q \oplus d) \quad (2)$$

donde $\oplus$ denota la concatenación del par consulta-documento. A diferencia de los *embeddings* bi-encoder, el *cross-encoder* modela la interacción completa entre consulta y documento en una única pasada, produciendo puntuaciones más precisas a costa de mayor latencia.

IV-D. Diversificación MMR

Para evitar fragmentos redundantes en la respuesta, se aplica Maximal Marginal Relevance [?]. Dado el conjunto de candidatos $\mathcal{F}$ y el subconjunto de documentos ya seleccionados $\mathcal{S}$, el siguiente fragmento elegido es:

$$d^* = \arg \max_{d \in \mathcal{F} \setminus \mathcal{S}} \left[\lambda \cdot \text{sim}(d, \mathbf{q}) - (1 - \lambda) \cdot \max_{d' \in \mathcal{S}} \text{sim}(d, d') \right] \quad (3)$$

donde $\lambda \in [0, 1]$ equilibra la relevancia respecto a la consulta y la diversidad respecto a los documentos ya seleccionados. El valor por defecto $\lambda = 0.5$ trata ambos criterios simétricamente.

IV-E. Pipeline Completo de Recuperación

El orden de aplicación de las etapas de post-proceso es:

1. **Reranker** sobre los primeros N_r candidatos densos (si habilitado).
2. **MMR** sobre el pool resultante (si habilitado).
3. **Top- K final**: se retornan los K fragmentos con mayor puntuación.

La salida es un objeto `SalidaRecuperacionRagDenso` con dos campos clave: `respuesta_contexto` (texto concatenado de los fragmentos, con cabeceras de identificación) y `fuentes` (lista de `FuenteRagDenso` con `score_denso`, `score_final` y metadatos de trazabilidad).

V. ORQUESTACIÓN CON LANGGRAPH

V-A. Estado Compartido

El grafo opera sobre un estado tipado `EstadoAgente` (`TypedDict` de Python), que actúa como memoria de corto plazo dentro de un turno. Los campos clave se muestran en la Tabla ??.

El campo `pensamientos` usa el *reducer* `operator.add`, que agrega (en lugar de sobrescribir) los eventos de razonamiento producidos por cada nodo, permitiendo transmitirlos progresivamente al cliente.

V-B. Nodos y Flujo del Grafo

El grafo contiene seis nodos secuenciales sin aristas condicionales explícitas. Las decisiones de ramificación se toman dentro de cada nodo mediante lógica condicional interna. La Figura ?? muestra el flujo completo.

Cuadro III
CAMPOS PRINCIPALES DEL ESTADO DEL GRAFO LANGGRAPH

Campo	Tipo	Descripción
<code>pregunta</code>	<code>str</code>	Entrada del usuario
<code>session_id</code>	<code>str</code>	ID de sesión canónico
<code>mensajes_historial</code>	<code>list[BaseMsg]</code>	Ventana de memoria
<code>intencion</code>	<code>str</code>	factual / listado / conteo
<code>tool_decidida</code>	<code>str None</code>	Herramienta elegida
<code>resultado_tool</code>	<code>dict</code>	Salida de la herramienta
<code>fuentes</code>	<code>list[dict]</code>	Chunks RAG (si aplica)
<code>pensamientos</code>	<code>list[dict]</code>	Traza del router (SSE)
<code>respuesta_final</code>	<code>str</code>	Texto del compositor

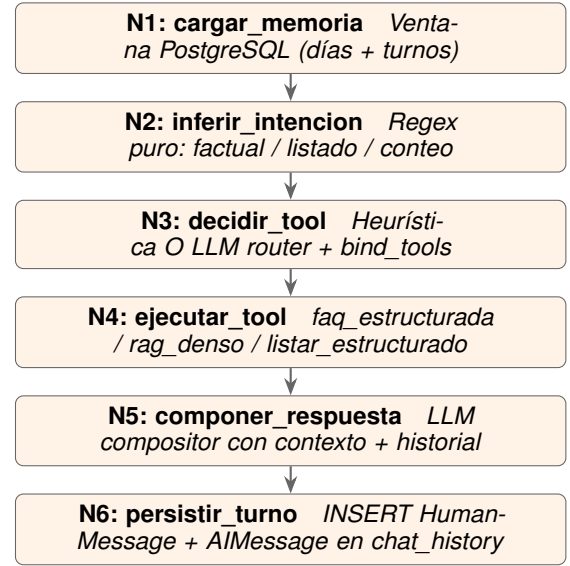


Figura 3. Grafo LangGraph del agente: seis nodos secuenciales. Las decisiones de herramienta se toman dentro del nodo N3 y el fallback a RAG desde N4 si el listado no retorna resultados.

V-B1. Nodo N1 — cargar_memoria: Carga la ventana de historial del usuario desde PostgreSQL filtrando por fecha (`HISTORIAL_DIAS_MAX`, defecto 7 días) y número de turnos (`HISTORIAL_TURNOS_MAX`, defecto 20), usando `PostgresChatMessageHistory` de `langchain-postgres`.

V-B2. Nodo N2 — inferir_intencion: Clasifica la consulta sin invocar al LLM mediante expresiones regulares:

- **conteo:** patrones como “cuántos”, “cantidad de”.
- **listado:** patrones como “lista”, “enumera”.
- **factual:** caso por defecto (resto de consultas).

Esta clasificación local reduce la latencia en rutas predecibles a menos de 1 ms.

V-B3. Nodo N3 — decidir_tool: Si la intención es *listado* o *conteo*, extrae filtros de *payload* mediante heurísticas (especialidades, sedes, tipo de página) y genera directamente el *tool call* a `listar_estructurado` sin consultar al LLM. En caso contrario, invoca al LLM router con `bind_tools` sobre las definiciones de herramientas del meta-prompt. El *fallback* predeterminado cuando el router no emite *tool_calls* es `rag_denso`.

V-B4. *Nodo N4 — ejecutar_tool*: Invoca la herramienta decidida y captura su salida. Si `listar_estructurado` retorna `conteo = 0`, se ejecuta un *fallback* automático a `rag_denso` con la pregunta original, garantizando que el usuario reciba siempre una respuesta contextualizada.

V-B5. *Nodo N5 — componer_respuesta*: Construye el mensaje al LLM compositor con: instrucciones institucionales, nombre del usuario (trato personalizado), historial previo y el contexto JSON serializado de la herramienta. La respuesta se transmite en *streaming* token a token al cliente.

V-B6. *Nodo N6 — persistir_turno*: Almacena el par pregunta-respuesta en `chat_history` (PostgreSQL) con metadatos de la herramienta utilizada y las fuentes RAG si aplican.

VI. LAS TRES HERRAMIENTAS DEL AGENTE

VI-A. FAQ Estructurada

La herramienta `faq_estructurada` responde consultas frecuentes sin necesidad de LLM ni de Qdrant, con latencia inferior a 5ms. Cada entrada del archivo `data/structured/faqs.json` contiene: `id`, `intent`, `keywords`, `pregunta_canonica` y `respuesta`.

La función de puntuación combina dos señales:

$$s_{kw} = \frac{|\{k \in \text{keywords} : \text{tokens}(k) \subseteq \text{tokens}(q)\}|}{|\text{keywords}|}$$

$$s_{can} = \frac{|\text{tokens}(p_c) \cap \text{tokens}(q)|}{|\text{tokens}(p_c)|}$$

$$s_{efec} = \max(s_{kw}, s_{can})$$

donde q es la consulta, p_c es la pregunta canónica y los `tokens` se normalizan (NFKD + minúsculas, sin stopwords). Si $s_{efec} \geq \tau_{faq}$ (umbral por defecto = 0.6), se retorna la respuesta asociada; en caso contrario el router elige otra herramienta.

Un mecanismo de caché basado en `mtime` del archivo JSON permite actualizar las FAQs sin reiniciar el servidor.

VI-B. RAG Denso

Implementa el pipeline de recuperación descrito en la Sección ???. La herramienta acepta los parámetros `consulta`, `filtros_tipo_pagina` (opcionales) y parámetros RAG inyectados desde el `RuntimeAgenteBundle`. El contexto retornado es un bloque de texto con fragmentos delimitados y sus fuentes.

VI-C. Listar Estructurado

Realiza un *scroll* sobre el *payload* de Qdrant con filtros exactos (`tipo_pagina`, `especialidad`, `sedes`), sin computar similitud vectorial. Esto permite responder preguntas de agregación como “¿cuántos pediatras tienen en Cali?” con el conteo exacto de puntos que cumplen los filtros, en lugar de aproximaciones semánticas. La salida incluye los campos `conteo`, `items` (lista de registros deduplicados), `muestra_truncada` y `filtros_aplicados`.

La Tabla ?? compara las tres herramientas.

Cuadro IV
COMPARACIÓN DE LAS TRES HERRAMIENTAS DEL AGENTE

Herramienta	Backend	LLM	Latencia	Caso de uso
<code>faq_estr.</code>	JSON curado	No	<5 ms	Preguntas frecuentes
<code>rag_denso</code>	Qdrant	No	50–300 ms	Búsqueda semántica
<code>listar_estr.</code>	Qdrant scroll	No	30–200 ms	Conteo y listados

VII. MEMORIA CONVERSACIONAL EN POSTGRESQL

VII-A. Esquema de Base de Datos

El sistema mantiene tres tablas en PostgreSQL administradas con SQLAlchemy 2 *async* y Alembic. La Tabla ?? resume su propósito.

Cuadro V
TABLAS OLTP DEL SISTEMA

Tabla	Gestión	Propósito
<code>usuarios</code>	Alembic	Registro de usuarios con UUID
<code>chat_history</code>	LangChain	Historial de mensajes por sesión
<code>config_admin_m2</code>	Alembic	Configuración hot-reload (singleton)

La tabla `usuarios` almacena `id` (UUID), `documento_identidad` (único), `nombre` y `marcas de tiempo`. La tabla `chat_history` es creada en *runtime* por `PostgresChatMessageHistory.create_tables` y almacena cada mensaje como un objeto JSONB con tipo, contenido y metadatos (herramienta usada, fuentes RAG).

VII-B. Ventana de Memoria y Control Optimista

La ventana de memoria se configura con dos parámetros independientes: `HISTORIAL_DIAS_MAX` (defecto 7 días) y `HISTORIAL_TURNOS_MAX` (defecto 20 turnos). Ambas restricciones se aplican en la misma consulta SQL, garantizando que el contexto de historial no desborde la ventana del LLM.

La tabla `config_admin_m2` implementa *locking* optimista mediante un campo `version`: el endpoint `PATCH /api/admin/config` solo actualiza si la versión recibida coincide con la almacenada, retornando HTTP 409 en caso de conflicto.

VIII. API HTTP Y PROTOCOLO SSE

VIII-A. Autenticación Liviana

El endpoint `POST /api/sesiones` recibe `documento_identidad` y `nombre`, crea o recupera el registro en `usuarios`, y emite una cookie HTTP-only con el `session_id` (`user:{uuid}`). Las peticiones posteriores se autentican mediante la cookie, la cabecera `X-Session-Id` o el parámetro `session_id` (en ese orden de precedencia).

VIII-B. Streaming SSE Extendido

El endpoint `POST /api/agente/stream` retorna un `EventSourceResponse` con seis tipos de eventos (Tabla ??). El cliente React los procesa progresivamente: primero muestra el razonamiento del router (*pensamiento*), luego los metadatos de la herramienta ejecutada (*herramienta*), seguido de los tokens del compositor en tiempo real (*token*), las fuentes RAG (*fuentes*, si aplica) y finalmente el evento de cierre (*final*).

Cuadro VI
EVENTOS SSE DEL AGENTE CONVERSACIONAL

Evento	Cuándo se emite	Payload clave
<i>pensamiento</i>	Tras N3 (decidir_tool)	herramienta_candidata, razón
<i>herramienta</i>	Tras N4 (ejecutar_tool)	nombre, latencia_ms, resultados
<i>token</i>	Por cada token del N5	motor, texto
<i>fuentes</i>	Si RAG retorna chunks	lista de FuenteRagDenso
<i>final</i>	Al finalizar N6	texto, latencia_ms, modelo
<i>error</i>	Si excepción en el grafo	código, mensaje

Un *watchdog* asíncrono monitorea la desconexión del cliente cada 250ms y aborta el streaming si este se desconecta, evitando cómputo innecesario en el servidor.

VIII-C. Panel Administrativo y Hot-Reload

El router `/api/admin` (protegido por la cabecera `X-Admin-Key`) expone los endpoints de la Tabla ?. El mecanismo de *hot-reload* funciona así: el endpoint `PATCH /api/admin/config` actualiza la tabla `config_admin_m2`; la siguiente petición SSE lee la configuración actualizada y reconstruye el `RuntimeAgenteBundle` sin reiniciar Uvicorn. Los parámetros ajustables incluyen: modelo LLM del router y compositor, temperatura, top-p, parámetros RAG (top-k, score mínimo, MMR, reranker) y los prompts institucionales.

Cuadro VII
ENDPOINTS DEL PANEL ADMINISTRATIVO

Método	Ruta	Acción
GET	<code>/api/admin/config</code>	Configuración efectiva fusionada
PATCH	<code>/api/admin/config</code>	Actualización parcial (hot-reload)
GET	<code>/api/admin/usuarios</code>	Listado paginado (doc_id enmascarado)
GET	<code>/api/admin/metricas</code>	Agregados: sesiones, total chats

IX. INTERFAZ DE USUARIO: REACT + VITE

IX-A. Migración desde Gradio

El Módulo 1 empleaba Gradio para la interfaz de usuario. En el Módulo 2, la interfaz se migró a una SPA (*Single Page Application*) desarrollada con React 19 y Vite 8, que ofrece mayor control sobre la experiencia de usuario, soporte completo de TypeScript 6 estricto, componentes accesibles con shadcn/ui [?] sobre Radix UI y soporte nativo de modo oscuro/claro con next-themes.

IX-B. Cliente SSE Personalizado

El `EventSource` nativo del navegador no soporta peticiones `POST` con cuerpo JSON. Para mantener la semántica RESTful del endpoint, se implementó un cliente SSE propio en `frontend/src/lib/sseClient.ts` usando la combinación `fetch + ReadableStream + TextDecoderStream`:

```
1 async function* streamSseEvents(url, body, headers)
2 {
3   const res = await fetch(url, {
4     method: "POST",
5     headers: { "Content-Type": "application/json",
6               ...headers },
7     body: JSON.stringify(body),
8     credentials: "include",
9   });
10  const reader = res.body
11    .pipeThrough(new TextDecoderStream())
12    .pipeThrough(new EventStreamTransformStream())
13    .getReader();
14  while (true) {
15    const { done, value } = await reader.read();
16    if (done) break;
17    yield { tipo: value.event,
18           payload: JSON.parse(value.data) };
19  }
```

Listing 1. Esquema del cliente SSE personalizado

Los eventos se validan con esquemas Zod para garantizar tipado seguro en tiempo de ejecución. El proxy de Vite en desarrollo reenvía `/api/*` al backend en `localhost:8000`, eliminando la necesidad de configurar CORS durante el desarrollo.

IX-C. Componentes Principales

- **IdentificationPage**: formulario de documento e identidad antes del chat; llama `POST /api/sesiones` y almacena el contexto de sesión.
- **ChatPage + MessageList**: historial de mensajes con renderizado Markdown (react-markdown + remark-gfm) y resaltado de sintaxis (shiki). Las fuentes RAG se muestran como tabla colapsable.
- **AdminDashboardPage**: panel de configuración hot-reload con selectores de modelo, sliders de temperatura/top-p, toggles de MMR y reranker, y editores de prompts.
- **ThemeToggle** + modo oscuro/claro: persiste en `localStorage` y respeta la preferencia del sistema operativo.

X. DESPLIEGUE CON DOCKER COMPOSE

El archivo `docker-compose.yml` define los servicios de la Tabla ?. El servicio `db-init` ejecuta `alembic upgrade head` como condición previa al arranque de la API, garantizando que las migraciones se apliquen antes de aceptar tráfico.

El *build* multi-stage compila el frontend React y lo sirve como archivos estáticos desde FastAPI, eliminando la necesidad de un servidor web adicional en producción. Los datos persisten en volúmenes nombrados `postgres-data` y `qdrant-storage`.

Cuadro VIII
SERVICIOS DEL STACK DOCKER COMPOSE

Servicio	Imagen	Puerto (host)	Rol
postgres	postgres:17.9-alpine	15432	OLTP + memoria
qdrant	qdrant/qdrant:v1.18.0	6333, 6334	Vector store
pgweb	sosedoff/pgweb:0.16.2	8081	UI admin Postgres
db-init	(imagen API)	—	Migraciones Alembic
api	(multi-stage build)	8000	FastAPI + frontend

XI. EVALUACIÓN

XI-A. Dataset de Referencia

Para la evaluación del módulo RAG se construyó un *golden set* en formato JSONL almacenado en `data/eval/golden_set_rag.jsonl`. Cada entrada contiene la consulta, el archivo fuente esperado y, opcionalmente, el fragmento de texto de referencia. Adicionalmente, el archivo `tests/qa/preguntas_evaluacion.yml` (23 preguntas del Módulo 1) permanece como referencia de cobertura temática.

XI-B. Métricas de Recuperación

El script `scripts/eval_metricas_rag.py` calcula las métricas de la Tabla ?? para los K fragmentos recuperados.

Cuadro IX
MÉTRICAS DE EVALUACIÓN DEL SISTEMA RAG

Métrica	Granularidad	Definición
Hit@K	Chunk	1 si algún chunk top-K coincide con ground truth
Precision@K	Doc	$ \text{docs recuperados} \cap \text{gt} / K$
Recall@K	Doc	$ \text{docs recuperados} \cap \text{gt} / \text{gt} $
MRR	Chunk	1/rango del primer acierto
nDCG@K	Ranking	$\text{DCG}@K / \text{iDCG}@K$

Donde:

$$\text{DCG}@K = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)} \quad (4)$$

$$\text{iDCG}@K = \sum_{i=1}^{\min(K, |\text{gt}|)} \frac{1}{\log_2(i+1)} \quad (5)$$

siendo $\text{rel}_i \in \{0,1\}$ si el i -ésimo fragmento recuperado pertenece al conjunto de referencia.

XI-C. Escenarios de Prueba E2E

La suite `tests/e2e/test_escenarios_modulo2.py` cubre cuatro escenarios con `MOCK_LLM=1` (router determinista) para verificar el comportamiento del grafo sin dependencia de la API de OpenAI (Tabla ??).

XI-D. Resultados

La Tabla ?? presentará los resultados cuantitativos de recuperación y generación una vez ejecutado el protocolo sobre el corpus definitivo.

Nota: los valores se completarán con los resultados del golden set definitivo ejecutado sobre el corpus de producción.

Cuadro X
ESCENARIOS E2E DEL MÓDULO 2

Escenario	Herramienta	Verificación
A — RAG	<code>rag_denso</code>	Evento <i>fuentes</i> con chunks
B — Memoria	<code>rag_denso</code> × 2	Historial persiste entre turnos
C — FAQ	<code>faq_estructurada</code>	<code>faq_match</code> <code>_encontrado=true</code>
D — Mixto	FAQ + RAG + FAQ	Secuencia y estado coherentes

Cuadro XI
RESULTADOS DE EVALUACIÓN RAG POR CONFIGURACIÓN

Configuración	Hit@5	Recall@5	nDCG@5	Latencia (ms)
Solo denso (base)	—	—	—	—
Denso + MMR	—	—	—	—
Denso + Reranker	—	—	—	—
Denso + Reranker + MMR	—	—	—	—

XII. DISCUSIÓN

XII-A. Comparación con el Módulo 1 (BM25)

La migración de BM25 a recuperación densa representa un cambio cualitativo en la capacidad de comprensión semántica del sistema. BM25 requería coincidencia léxica exacta; el sistema de Módulo 2 puede recuperar documentos relevantes ante consultas con sinónimos, abreviaciones o paráfrasis. Sin embargo, este avance tiene un costo: la recuperación densa depende de un proveedor de *embeddings* (en la configuración por defecto, la API de OpenAI), lo que introduce una dependencia externa y un costo por token. La opción de HuggingFace local mitiga esta dependencia a expensas de mayor latencia de ingesta.

XII-B. Ventajas del Diseño con Tres Herramientas

La segmentación en `faq_estructurada`, `rag_denso` y `listar_estructurado` responde a una observación empírica: distintos tipos de consultas tienen perfiles de información óptimos. Las preguntas frecuentes (horarios, contacto, datos institucionales) son mejor respondidas por respuestas curadas y deterministas. Las preguntas conceptuales o clínicas se benefician de la recuperación semántica. Los listados y conteos requieren exactitud sobre el conjunto de datos completo, no aproximaciones.

XII-C. Limitaciones

- Sincronización corpus-vectores:** cambios en `data/markdown/` requieren reindexación explícita para reflejarse en Qdrant.
- Latencia del reranker en CPU:** el modelo `BAAI/bge-reranker-base` puede tardar 300–800 ms por turno en CPU, lo que lo hace inadecuado en producción sin GPU.
- Costo de API:** la inferencia del router y los embeddings dependen de OpenAI en la configuración por defecto.

4. **Retención de datos:** el historial conversacional contiene datos personales; la ventana temporal configurable mitiga la acumulación indefinida pero no sustituye una política de privacidad formal.

XIII. CONCLUSIONES

Este trabajo presentó un agente conversacional completo para la Fundación Valle del Lili que supera las limitaciones del sistema BM25 del Módulo 1 mediante seis contribuciones técnicas:

1. **RAG denso con pipeline configurable:** similitud coseno en Qdrant, filtro por umbral, reranking con *cross-encoder* y diversificación MMR, todo controlable desde variables de entorno o el panel administrativo en tiempo de ejecución.
2. **Orquestación con LangGraph:** grafo de seis nodos secuenciales con clasificación de intención mediante regex (sin LLM), selección de herramienta por heurísticas o *tool-calling*, y mecanismo de *fallback* encadenado.
3. **Tres herramientas complementarias:** FAQ determinista, RAG semántico y listado estructurado por *payload*, cubriendo los principales patrones de consulta institucional.
4. **Memoria persistente:** ventana de historial en PostgreSQL con control temporal y de turnos, integrada de forma transparente en el estado del grafo.
5. **API SSE con seis tipos de eventos:** trazabilidad completa del razonamiento del agente expuesta al cliente, desde la decisión de herramienta hasta las fuentes RAG.
6. **Panel administrativo con hot-reload:** ajuste de parámetros LLM, RAG y prompts sin reiniciar el servidor, con control de versión optimista para entornos con múltiples operadores.

La arquitectura demuestra que es posible construir un sistema de asistencia conversacional institucional robusto, trazable y ajustable en producción, combinando componentes de código abierto (LangGraph, LlamaIndex, Qdrant, PostgreSQL, React) con modelos de lenguaje de acceso comercial.

REFERENCIAS

- [1] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] S. Robertson y H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, n.º 4, pp. 333–389, 2009.
- [3] H. Chase, “LangChain: Building applications with LLMs through composability,” *GitHub repository*, 2022. [En línea]. Disponible en: <https://github.com/langchain-ai/langchain>
- [4] LangChain AI, “LangGraph: Build stateful, multi-actor applications with LLMs,” 2024. [En línea]. Disponible en: <https://github.com/langchain-ai/langgraph>
- [5] Qdrant, “Qdrant: Vector Database for the next generation of AI applications,” 2024. [En línea]. Disponible en: <https://qdrant.tech>
- [6] J. Carbonell y J. Goldstein, “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries,” en *Proc. 21st ACM SIGIR*, pp. 335–336, 1998.
- [7] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, “C-Pack: Packaged Resources To Advance General Chinese Embedding,” *arXiv preprint arXiv:2309.07597*, 2023.
- [8] LangChain AI, “langchain-postgres: PostgreSQL integration for LangChain,” 2024. [En línea]. Disponible en: <https://github.com/langchain-ai/langchain-postgres>

- [9] J. Liu, “LlamaIndex: Data Framework for LLM Applications,” *GitHub repository*, 2022. [En línea]. Disponible en: https://github.com/run-llama/llama_index
- [10] OpenAI, “GPT-4o System Card,” 2024. [En línea]. Disponible en: <https://openai.com/index/gpt-4o-system-card>
- [11] S. Ramírez, “FastAPI: Modern, fast (high-performance) web framework for building APIs,” 2024. [En línea]. Disponible en: <https://fastapi.tiangolo.com>
- [12] shadcn, “shadcn/ui: Beautifully designed components built with Radix UI and Tailwind,” 2024. [En línea]. Disponible en: <https://ui.shadcn.com>